



RESEARCH ENGINEER

Take-home task

- This task has two parts: make two fixed small models collaborate to solve math problems and prove the answers in Lean 4, then understanding how collaboration improves performance.
- Part one is scored mechanically on a private holdout set: points only for proofs the Lean compiler accepts. We clone your fork and run it ourselves.
- Submit by **August 24** (EOD Anywhere on Earth) by uploading your writeup as a **PDF** to the Google form shared with this task; your repository link goes in the same form.

Background

Our autoresearch framework has LLM agents find theorems about the internal computations of transformers and verify them in Lean 4. We believe we can get more from models when they work together, and that unlocking this potential depends on finding the right way for them to collaborate. This task recreates the setting at a smaller scale with two lightweight models.

Everything you need lives in one repository:

github.com/VerifiedMechanisms/re-takehome

- The kit contains ten development problems, a working single-model baseline, scripts that set up Lean and Mathlib for you, and the one-page rules in [RULES.md](#).
- Your OpenRouter API key arrives in the same email as this PDF: a **\$50 budget, on us**, for the two models named in the rules.
- Build with whatever you like, including your own chatbots and coding agents; at run time your system talks only to the two fixed models through our key.

PART ONE

Make two models prove things

Each problem is a folder: `problem.md` states it in English, `challenge.lean` states it formally, with `sorry` where the proof (and on some problems a numeric answer) goes. Your system turns the folder into a Lean file the compiler accepts, with our statement unchanged.

Build the coordination layer. Both models must take part in every scored attempt; how you divide the work between them is the design problem. We would rather see a simple design we can fully understand than a complicated one that scores marginally better; if two systems score about the same, we prefer the simpler one.

Scoring is mechanical, on a private holdout with the dev set's format and grading: about a dozen problems whose difficulty may vary, one point per correct problem, zero for incorrect. A proof counts only if it passes the open-source Lean comparator ([leanprover/comparator](#)): it must compile against our pristine statement with no `sorry` and no axioms beyond the standard three. A problem that spends more than \$1 scores zero even if the proof is correct. We run your repository once per problem at 20 minutes and \$1 each.

PART TWO

The eval science of collaboration

Is your two-model system better than either model alone, and if so, where exactly does one model fill the other's gaps? Run three conditions on the dev set or your own set of problems: **model A solo, model B solo, and your collaboration**.

Report per problem (which problems flipped between conditions), while accounting for confounding factors as much as possible. Classify every failed attempt: wrong informal solution; right solution but failed formalization; proof repair stuck; harness error. Include at least two quoted transcript case studies, the holdout score you expect with a sentence of reasoning, and a spend ledger.

Instructions

- **What should the writeup look like?** Two to ten pages, appendix excluded. The repository carries the code; the PDF explains design choices, results and the writeup for the eval science of collaboration.
- **Rules.** Read [RULES.md](#) before building.
- **Using LLMs.** Use LLM assistance freely for the harness and the writeup (Codex, Claude Code, whatever you normally reach for); disclose which models and tools you used and how in the Google Form.

If you have any questions, please write to hiring@verifiedmechanisms.ai.